NRLC-08: Validation, Diff & Rollback

> **Series:** NRLC — New Relic to Dynatrace Migration Deep Dives | **Notebook:** 8 of 9 | **Created:** April 2026 | **Last Updated:** 10/19/2026

Overview

Validation is what separates a real migration from "we hope this works." This deep dive covers DQL syntax validation, live tenant validation against the DT Environment Registry, diff against existing DT configuration, the rollback manifest pattern, and conversion-quality reports. These are the safety nets the migration runs through before each cutover gate.

Table of Contents

1. Three-Tier Validation Model
2. DQL Syntax Validation
3. Live Tenant Validation (DTEnvironmentRegistry)
4. Behavioral Validation (Dual Run)
5. Diff Against Live Config
6. Rollback Manifests
7. Conversion Quality Reports

Prerequisites

Requirement	Details
Audience	Migration leads, QA engineers, SRE doing cutover sign-off
Tooling	`Dynatrace-NewRelic` validators + `migration/` modules; `nrql-engine`'s `DQLSyntaxValidator` and `DTEnvironmentRegistry`

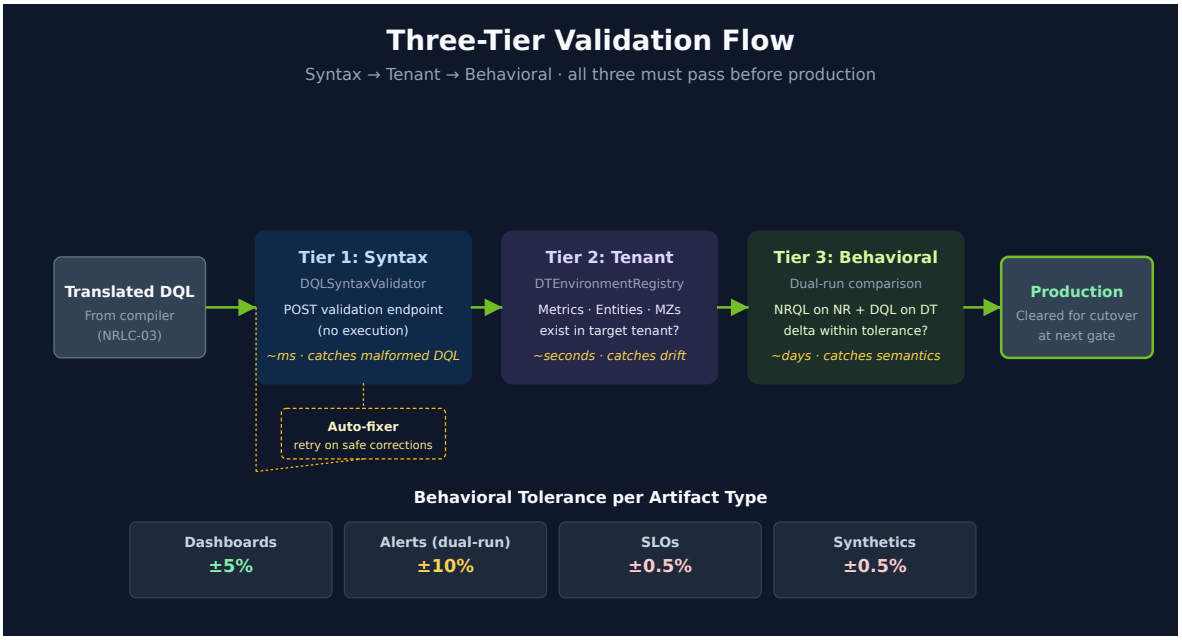
Requirement	Details
Required reading	NRLC-02 (validation depends on the compiler's confidence scoring)

1. Three-Tier Validation Model

Every migrated artifact passes through three tiers of validation:

Tier	What It Checks	Tool	Speed	Catches
Tier 1: Syntax	DQL parses; no malformed JSON	`DQLSyntaxValidator`	Fast	Compiler bugs, malformed translations
Tier 2: Tenant	Referenced metrics, entities, buckets, enrichment attributes exist	`DTEnvironmentRegistry`	Medium	Environment-specific drift, missing prerequisites
Tier 3: Behavioral	Same data shape and magnitudes as NR	Dual-run + DQL diff	Slow (days)	Semantic translation errors

All three tiers must pass before an artifact is production-ready. Skipping Tier 3 is the most common mistake — syntactically valid DQL can still produce wrong numbers.



2. DQL Syntax Validation

The `DQLSyntaxValidator` posts each DQL query to the Grail language service's validation endpoint without executing it:

```
POST /platform/storage/query/v1/query:verify
body: { query: "&lt;DQL&gt;" }
```

This is the dedicated Grail language-service endpoint for DQL syntax validation (sibling endpoints `/query:parse` and `/query:autocomplete` cover parsing and autocomplete) — it is not a Settings 2.0 object write, since posting a DQL string embedded in a Settings object would validate the object's schema, not the query itself. DT responds with parser errors if the DQL is invalid. Common issues caught:

- Unknown DQL functions
- Misuse of named parameters (e.g., `round(value, 2)` instead of `round(value, decimals: 2)`)
- Array-vs-scalar type errors on `timeseries` results
- Reserved keyword as field name (must be backtick-quoted)
- Missing time range on `fetch` (warning, not error)

Auto-Fix Loop

When syntax validation fails, the engine attempts safe corrections via `DQLFixer`:

1. Try applying known fix patterns (named parameters, operator normalization, etc.)
2. Re-validate
3. If still failing, return failure with original DQL + attempted fix + error message

Applied fixes are returned in `result.fixes[]` for transparency.

3. Live Tenant Validation (DTEnvironmentRegistry)

Syntax validation only confirms the DQL parses. Tenant validation confirms the DQL refers to things that exist in *your* tenant.

The `DTEnvironmentRegistry` lazy-loads tenant inventory:

Lookup	Source API	Validates
Metric keys (`dt.host.cpu.usage`, etc.)	`GET /api/v2/metrics` — on a latest-Dynatrace tenant, prefer the DQL `metrics` command	Metric exists; displayName, unit
Entity types and IDs	`GET /api/v2/entities` — on a latest-Dynatrace tenant, prefer `smartscapeNodes` (FAQ entry 16)	Entity present; resolves names ↔ IDs
Existing dashboards	`GET /platform/document/v1/documents`	Avoids duplicate creation
OpenPipeline enrichment rules	`GET /api/v2/settings/objects?schemaIds=builtin:openpipeline.logs.pipelines` (and `.routing`, `.ingest-sources` per scope)	Enrichment rule exists; produces expected attribute values on ingested data
Buckets	`GET /platform/storage/management/v1/bucket-definitions`	Bucket exists with expected retention; routing rule targets it correctly
Synthetic locations	`GET /api/v2/synthetic/locations` (platform alternative, early adopter: `GET /platform/synthetic/v1/locations`, `synthetic:locations:read`)	NR location → DT location lookup
Notifications	`GET /api/v2/settings/objects?schemaIds=builtin:problem.notifications` — the **schema** is blocked at upgrade (the `objects` endpoint is not); workflow-based notifications replace it	No duplicate notification objects

> **Note — OpenPipeline API deprecation:** The direct `GET/PUT /platform/openpipeline/v1/configurations` endpoints were deprecated in SaaS 1.327 and reached end of life on **June 29, 2026**. Read and manage OpenPipeline configuration through Settings 2.0 with the `builtin:openpipeline.<scope>.routing`,

> `builtin:openpipeline.<scope>.pipelines`, and `builtin:openpipeline.<scope>.ingest-sources` schemas. See the official API deprecation guide.

Common failures caught here:

- Migrated alert references `dt.host.cpu.usage` but tenant only ingests `dt.host.cpu.user` (and OOTB metric naming changed in DT version)
- DQL attribute filter references an attribute value not produced by any OpenPipeline enrichment (rule not yet applied or typo)
- Entity name lookup returns multiple matches — ambiguous selector
- Synthetic location `AWS_US_EAST_1` not enabled in target tenant

4. Behavioral Validation (Dual Run)

Both platforms produce numbers from the same underlying world. Validate they agree.

Dual-Run Pattern

1. Configure NR and DT both ingesting from the same source (during the dual-run window)
2. For each migrated artifact, run the original NRQL against NR and the translated DQL against DT for the **same time window**
3. Compare the result shape and magnitudes
4. Investigate any discrepancy > acceptable tolerance (typically 1–5% depending on artifact type)

Comparison Tooling

The `Dynatrace-NewRelic` CLI has **no** NR-vs-DT result comparison. Its `migrate.py audit` command is a *configuration* drift check: it compares the transformed baseline a migration run wrote against the live Dynatrace tenant and reports `RENAMED`, `DELETED`, `MODIFIED` and `EXTRA` entities. It is read-only, exits 1 when it finds drift, and never queries New Relic.

```
# Config drift: live tenant vs the transformed baseline from a migration run
python3 migrate.py audit --baseline ./output/transformed/dynatrace_config.json --
output drift.json
```

The dual-run comparison above is therefore manual: run each NRQL query in New Relic and its DQL translation in Dynatrace over the same window, record both values, and compare them against the tolerances below.

Tolerance by Artifact Type

Artifact	Tolerance
Dashboard count widgets	±5% (display)
Alert thresholds	±10% during dual-alert tuning
SLO SLI values	±0.5%
Synthetic availability	±0.5%
Log volumes	±5%

5. Diff Against Live Config

Before importing migrated artifacts, diff them against what already exists in the target DT tenant. This avoids:

- Duplicating dashboards that already exist (e.g., from a previous partial migration)
- Overwriting tenant admin's customized Workflows
- Recreating notifications when an existing one matches

```
# --dry-run is what keeps this read-only: without it, --diff runs after a full
import
python3 migrate.py migrate --dry-run --diff --components dashboards,alerts --
output ./output
```

Output classifies each migrated entity (actions as defined in the tool's `migration/diff.py`, read 10/05/2026):

Action	Meaning
`CREATE`	No match found in DT — will be created. Also the result for every entity type the diff cannot look up
`UPDATE`	An existing DT entity has the same name
`CONFLICT`	Counted in the summary, but the current diff logic never assigns it
`ORPHAN`	Exists in DT but not in the NR export

The diff matches by name and only for **dashboards and management zones**; alerts (metric events, alerting profiles), SLOs and synthetic monitors are always reported as `CREATE`. It

cannot detect drift inside a matching dashboard or an alert that a previous run already migrated — check those by hand.

Always run `migrate --diff` with `--dry-run` before any full import. `--diff` is ignored with `--import-only`.

6. Rollback Manifests

Every migration run generates a **rollback manifest** — a JSON record of every entity created, with enough metadata to reverse the operation.

A full `migrate` run (not `--import-only`) writes it to `<output>/rollback-manifest.json`. Its shape, from the tool's `migration/state.py` (read 10/05/2026):

```
{
  "entries": [
    {
      "entity_type": "&lt;type&gt;",
      "dynatrace_id": "&lt;id&gt;",
      "name": "&lt;entity name&gt;",
      "timestamp": "&lt;ISO-8601 UTC&gt;"
    }
  ]
}
```

Rollback Command

```
python3 migrate.py migrate --rollback ./output/rollback-manifest.json
```

The rollback:

1. Reads the manifest and lists the target entities (with `--dry-run` it stops here)
2. Asks for confirmation
3. For each entity, calls the Dynatrace client's delete for that entity type, printing any failure
4. Prints a total of entities deleted and failed

Always retain manifests for at least the dual-run window + 30 days. Rollback at hour 71 of dual-run is rare but not theoretical.

7. Conversion Quality Reports

After each migration phase, generate a quality report:

```
python3 migrate.py migrate --report --output report.html
```

Report sections:

Section	Content
Summary	Counts: total / migrated / skipped / failed
By component	Per-type breakdown: dashboards N/M, alerts N/M, etc.
NRQL conversion	HIGH / MEDIUM / LOW counts; LOW queries listed
Failed translations	Each failure with reason and source query
Manual review queue	APM conditions, scripted browsers, LOW-confidence DQL
Rollback eligibility	Time-bounded list of recently-migrated entities

Reports are the migration's audit trail — attach them to your change-management ticket for each phase.

Summary

Validation is the migration's safety net. Three tiers — syntax, tenant, behavioral — must pass before any artifact is production-ready. Diff and rollback give you reversibility. Conversion-quality reports give you the audit trail.

Continue to **NRLC-09 Toolchain Reference & End-to-End Runbook**.

This notebook was AI-generated from community-submitted and publicly available sources, including the open-source NewRelic-to-Dynatrace-Migration-Utilities, nrql-engine (planned future home: the `dynatrace-dma` Dynatrace Migration Assistant organization), and nrql-translator projects. This notebook series is not officially supported by Dynatrace or New Relic. Always verify information against the official Dynatrace documentation and New Relic documentation.